ASSIGNMENT 06
OBJECT ORIENTED PROGRAMMING

Name : Mahak Goyal
Roll No. : 2026010066
Course : Object oriented programming with C++
Group : 1MCA3

1. **Create a class Counter with an integer data member count. Overload the unary ++ operator to increase the value of count by 1. Create an object and display its value before and after applying ++.**

```
#include <iostream>
using namespace std;
class Counter {
   int count;
public:
   Counter()   {
      count = 5;
   }
   void operator++()   {
      count++;
   }
   void display()   {
      cout << "Count = " << count << endl;
   }
};
int main() {
   Counter obj;
   cout << "Before increment: ";
   obj.display();
   ++obj;
   cout << "After increment: ";
   obj.display();
   return 0;
}
```

```
Before increment: Count = 5
After increment: Count = 6

--------------------------------
Process exited after 0.3455 seconds with return value 0
Press any key to continue . . .
```

2. **Create a class Number with an integer data member value. Overload the unary -- operator to decrease the value of the object by 1. Display the value before and after applying the operator.**

```
#include <iostream>
using namespace std;
class Number {
   int value;
public:
   Number()   {
      value = 10;
   }
   void operator--()   {
      value--;
   }
```

```
    void display()  {
      cout << "Value = " << value << endl;
   }
};
int main() {
   Number obj;
   cout << "Before decrement: ";
   obj.display();
   --obj;
   cout << "After decrement: ";
   obj.display();
   return 0;
}
```

```
Before decrement: Value = 10
After decrement: Value = 9

--------------------------------
Process exited after 0.2598 seconds with return value 0
Press any key to continue . . .
```

3. **Create a class Number containing an integer data member value. Create two objects and overload the binary + operator to add the values stored in the two objects. Display the result.**

```
#include <iostream>
using namespace std;
class Number {
   int value;
public:
   Number(int v)  {
      value = v;
   }
   Number operator+(Number obj)  {
      Number temp(value + obj.value);
      return temp;
   }
   void display()  {
      cout << "Value = " << value << endl;
   }
};
int main() {
   Number n1(10), n2(20), n3(0);
   n3 = n1 + n2;
   cout << "Sum: ";
   n3.display();
   return 0;
}
```

```
Sum: Value = 30

--------------------------------
Process exited after 0.2582 seconds with return value 0
Press any key to continue . . .
```

4. **Create a class Number containing an integer data member. Create two objects and overload the following operators:**
   + to add the values.
   - to subtract the values.
   Display the results of both operations.

```
#include <iostream>
using namespace std;
class Number {
   int value;
public:
   Number(int v)   {
      value = v;
   }
   Number operator+(Number obj)   {
      return Number(value + obj.value);
   }
   Number operator-(Number obj)   {
      return Number(value - obj.value);
   }
   void display()   {
      cout << value << endl;
   }
};
int main() {
   Number n1(20), n2(10), n3(0), n4(0);
   n3 = n1 + n2;
   n4 = n1 - n2;
   cout << "Addition = ";
   n3.display();
   cout << "Subtraction = ";
   n4.display();
   return 0;
}
```

```
Addition = 30
Subtraction = 10

--------------------------------
Process exited after 0.2367 seconds with return value 0
Press any key to continue . . .
```

5. **Create a class Complex with two data members real and imaginary. Overload the + operator to add two complex number objects and display the resulting complex number.**

```
#include <iostream>
using namespace std;
class Complex {
   int real, imaginary;
public:
   Complex(int r, int i)   {
      real = r;
      imaginary = i;
   }
```

```
    Complex operator+(Complex obj)    {
        return Complex(real + obj.real,
                       imaginary + obj.imaginary);
    }
    void display()    {
        cout << real << " + " << imaginary << "i" << endl;
    }
};
int main() {
    Complex c1(3, 4);
    Complex c2(2, 5);
    Complex c3(0, 0);
    c3 = c1 + c2;
    cout << "First number = ";
    c1.display();
    cout << "Second number = ";
    c2.display();
    cout << "Result = ";
    c3.display();
    return 0;
}
```

```
First number = 3 + 4i
Second number = 2 + 5i
Result = 5 + 9i

--------------------------------
Process exited after 0.232 seconds with return value 0
Press any key to continue . . .
```

6. **Create a class Student with a data member CGPA. Create two student objects and overload the following relational operators:**
   == to check whether both students have the same CGPA.
   > to check which student has the higher CGPA.
   Display an appropriate message based on the comparison.

```
#include <iostream>
using namespace std;
class Student {
    float CGPA;
public:
    Student(float c)    {
        CGPA = c;
    }
    bool operator==(Student obj)    {
        return CGPA == obj.CGPA;
    }
    bool operator>(Student obj)    {
        return CGPA > obj.CGPA;
    }
};
int main() {
    Student s1(8.5), s2(9.0);
    if (s1 == s2)
        cout << "Both students have the same CGPA." << endl;
```

```
    else
        cout << "Students have different CGPAs." << endl;
    if (s1 > s2)
        cout << "Student 1 has higher CGPA." << endl;
    else if (s2 > s1)
        cout << "Student 2 has higher CGPA." << endl;
    else
        cout << "Both students have the same CGPA." << endl;
    return 0;
}
```

